LP-III External Viva
Complete Questions and Short Answers

DAA | Machine Learning | Blockchain Technology
Prepared from your LP-III Lab Manual 2025-26 and Blockchain Lab Manual

★ **= question printed in your manual** (the external is most likely to ask these - learn them first). Questions without a star are the usual viva questions on the same topic.

Contents: Part A DAA (5 assignments) | Part B Machine Learning (5 assignments) | Part C Blockchain (6 assignments) | Part D Quick cheat sheet (complexities, formulas, comparisons) | Part E Mistakes in the manual

Tip: Answers are kept short - say them in your own words. Run each program once before the exam and remember your actual output.

PART A - DAA

Assignment 1 - Fibonacci (recursive and non-recursive)

★ 1. What is the Fibonacci sequence of numbers?

A series where every number is the sum of the previous two. It starts with 0 and 1: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

★ 2. How does the Fibonacci series work?

Start with F0 = 0 and F1 = 1. Then Fn = Fn-1 + Fn-2. Example: F9 = 34, F2 = 1.

★ 3. What is the Golden Ratio?

The ratio of two consecutive Fibonacci numbers (F(n+1)/F(n)) gets closer and closer to 1.618. This number is the Golden Ratio (phi = (1 + sqrt 5) / 2).

★ 4. What is the Fibonacci Search technique?

A search for sorted arrays. It splits the array using Fibonacci numbers instead of the middle point (binary search). It uses only add/subtract, time O(log n).

★ 5. What are real applications of the Fibonacci series?

Euclid's GCD run-time analysis, Fibonacci heap, Fibonacci search, cryptography and quantum mechanics, patterns in nature and music, Fibonacci retracement in stock trading.

6. What is the base condition of the recursive program?

If n <= 1, return n.

7. Explain the non-recursive program.

Keep n1 = 0 and n2 = 1. In a loop: print n1, nth = n1 + n2, then n1 = n2 and n2 = nth. Repeat for the number of terms.

8. Explain the recursive program.

recur_fibo(n) returns n if n <= 1, else returns recur_fibo(n-1) + recur_fibo(n-2).

9. Time and space of the recursive method?

Time $O(2^n)$ exponential. Space O(n) for the call stack.

10. Time and space of the non-recursive method?

Time O(n). Space O(1) with two variables (O(n) if an array is used).

11. Why is recursion slow?

It solves the same smaller problems again and again (overlapping subproblems).

12. How to speed it up?

Memoization or dynamic programming - store results once. Time becomes O(n).

13. Which is better - recursive or non-recursive?

Non-recursive. Faster, less memory, no stack overflow.

14. Can Fibonacci be done in O(log n)?

Yes - matrix exponentiation or fast doubling.

15. What is the recurrence relation for time?

T(n) = T(n-1) + T(n-2) + 1.

16. What is time complexity? What is space complexity?

Time complexity = how running time grows with input size. Space complexity = how memory grows with input size.

17. What is Big-O?

Upper bound (worst-case growth) of an algorithm.

18. Is the series zero based? Which term is F5?

Yes. F0 = 0, so F5 = 5 is the 6th term.

Assignment 2 - Huffman Encoding (Greedy)

★ 1. What is Huffman Encoding?

A lossless data compression technique by David Huffman. It gives variable-length codes: frequent characters get short codes, rare characters get long codes.

★ 2. How many bits are needed to encode "mississippi"?

Frequencies: i = 4, s = 4, p = 2, m = 1. Codes: s = 1 bit, i = 2 bits, m and p = 3 bits each. Total = 4x1 + 4x2 + 1x3 + 2x3 = **21 bits**. Without encoding: 11 x 8 = 88 bits.

★ 3. Which tree is used in Huffman encoding? Give an example.

A Huffman tree - a full binary tree built from frequencies; leaves are characters. Manual example codes: a = 111, e = 10, i = 00, o = 11001, u = 1101, s = 01, t = 11000.

★ 4. Why is Huffman coding lossless?

No data is thrown away. Codes are prefix-free, so the decoder gets back exactly the original text.

5. What is a greedy method?

Choose the best option available at the moment, never go back. May not always give the best result.

6. Why is Huffman a greedy algorithm?

Each step it merges the two lowest-frequency nodes - the best local choice.

7. What data structure is used?

Min-heap (priority queue).

8. Write the steps of Huffman coding.

1) Count frequencies. 2) Put characters in a min-heap as leaf nodes. 3) Take two minimum nodes, make a new node with their sum, smaller one left and next one right. 4) Put it back. 5) Repeat till one node is left. 6) Left edge = 0, right edge = 1.

9. What are the two major steps?

1) Build the Huffman tree. 2) Assign codes by walking from the root to each leaf.

10. What is the prefix rule?

No code is the beginning of another code. It avoids confusion while decoding.

11. How do you decode?

Read bits from the root: 0 = left, 1 = right. On reaching a leaf, output the character and restart from the root.

12. Time complexity?

O(n log n). extractMin is called 2(n-1) times and each takes O(log n).

13. Manual example: "BCCABBDDAECCBBAEDDCC" - how many bits?

Without encoding: 15 characters x 8 = 120 bits. With Huffman: 32 (data) + 15 (code table) + 28 (tree) = 75 bits.

14. Average code length formula?

Sum of (frequency x code length) / total frequency. (Manual example gives about 2.52 bits.)

15. Length of the encoded message formula?

Sum of (frequency x code length).

16. Is the Huffman code unique?

No, but the total length is always the minimum.

17. Fixed length vs variable length coding?

Fixed: same bits for every character (ASCII 8 bits). Variable: fewer bits for frequent characters.

18. Where is Huffman used?

ZIP, JPEG, MP3, PNG, fax.

Assignment 3 - Fractional Knapsack (Greedy)

★ 1. What is the Greedy approach?

Pick the best choice at each step without worrying about the future. Steps: start with empty solution, add an item, keep it if feasible, else reject it forever. Easy to design but not always optimal. Examples: Huffman, Kruskal, Prim, Dijkstra.

★ 2. Explain the concept of fractional knapsack.

A bag has limited capacity. Each item has value and weight. Items can be broken into pieces. Choose items/pieces to get maximum value.

★ 3. Difference between fractional and 0/1 knapsack?

Fractional: items divisible, solved by Greedy, O(n log n). 0/1: take whole item or nothing, solved by Dynamic Programming, O(nW).

★ 4. Solve one example other than the manual.

Values 25, 24, 15; weights 18, 15, 10; capacity W = 20. Ratios: 1.39, 1.6, 1.5. Take item 2 fully (15 kg, value 24). Remaining 5 kg of item 3: (5/10) x 15 = 7.5. **Total = 31.5**.

5. What are the steps?

1) Find value/weight for each item. 2) Sort in decreasing order of ratio. 3) Fill the bag from the top; take a fraction of the last item that does not fit.

6. Time complexity?

O(n log n) because of sorting. The loop is O(n).

7. Why does greedy work here but not for 0/1?

We can cut items, so the best-ratio items can fill the bag completely.

8. Manual example (capacity 60) answer?

Items I1, I2, I5 fully and 20/22 of I4. Value = 160 + (20/22) x 77 = 160 + 70 = **230**.

9. Code example answer (values 60,100,120; weights 10,20,30; W=50)?

240 = 60 + 100 + (20/30) x 120. (Manual prints 24 - typo.)

10. Why do we sort by ratio and not by value or weight?

Ratio shows value per kg, which is what the limited capacity should be spent on.

11. What is the knapsack problem?

Choose items to get maximum profit without crossing the weight limit.

12. What is the key line in your code?

arr.sort(key = value/weight, reverse = True), then add whole item if it fits, else add fraction value x W / weight and break.

Assignment 4 - Analysis of Quick Sort (Deterministic and Randomized)

1. What is quick sort?

A divide and conquer sorting algorithm. Pick a pivot, partition the array so smaller elements are on the left and larger on the right, then sort the two parts recursively.

2. What is a pivot?

The element used to split the array into two parts.

3. What is deterministic quick sort?

The pivot is chosen by a fixed rule (first, last or middle element). The same input always gives the same steps.

4. What is randomized quick sort?

The pivot is chosen at random from the current sub-array (random index, swapped with the last element, then normal partition).

5. Why do we use the randomized variant?

To avoid the worst case on sorted or nearly sorted input. No input is always bad, so the expected time is O(n log n) for every input.

6. Write the steps of quick sort.

1) If low >= high, stop. 2) Choose pivot. 3) Partition: smaller elements left, larger right, pivot in its final place. 4) Recursively sort the left part. 5) Recursively sort the right part.

7. Explain the partition procedure (Lomuto).

Take the last element as pivot. Keep i = low - 1. For j from low to high-1, if arr[j] <= pivot then i++ and swap arr[i], arr[j]. At the end swap arr[i+1] with the pivot and return i+1.

8. What changes in the randomized program?

Only one step: pick r = random.randint(low, high), swap arr[r] with arr[high], then call the same partition.

9. Example: partition [10, 7, 8, 9, 1, 5] with last element as pivot.

Pivot = 5. Elements smaller than 5: only 1. Result after partition: [1, **5**, 8, 9, 10, 7]. Pivot 5 is at index 1 (final place).

10. Time complexity - best, average, worst?

Best: O(n log n). Average: O(n log n). Worst: $O(n^2)$.

11. What is the best case?

Pivot always splits the array into two equal halves. T(n) = 2T(n/2) + n = O(n log n).

12. What is the worst case?

Pivot is always the smallest or largest element, so one part is empty. T(n) = T(n-1) + n = $O(n^2)$.

13. When does deterministic quick sort get its worst case?

With first or last element as pivot: when the array is already sorted, reverse sorted, or all elements are equal.

14. Does randomized quick sort have a worst case?

It can still happen, but only by very bad luck (probability extremely small). Its **expected** time is O(n log n) for every input.

15. What is the expected number of comparisons in randomized quick sort?

About 2 n ln n, which is about 1.39 n $\log_2$ n.

16. Why does randomization help?

Each element is equally likely to be the pivot, so a bad split is unlikely at every level. The running time no longer depends on the input order.

17. What is the time of the partition step?

O(n) - one pass through the sub-array.

18. Space complexity?

In-place. Extra space is the recursion stack: O(log n) on average, O(n) in the worst case.

19. Is quick sort stable? Is it in-place?

Not stable (equal elements may change order). Yes, in-place.

20. What is a Las Vegas algorithm? Which one is randomized quick sort?

A randomized algorithm that always gives the correct answer, only the running time varies. Randomized quick sort is Las Vegas. (Monte Carlo algorithms may give a wrong answer with small probability.)

21. How did you analyse the two variants in your program?

Ran both on the same inputs (sorted, reverse sorted, random) for different sizes n, and compared running time and/or number of comparisons. Deterministic is slow on sorted input; randomized stays close to n log n on all inputs.

22. What result do you expect on sorted input?

Deterministic (last/first pivot): $O(n^2)$, very slow. Randomized: O(n log n). On random input both are about the same.

23. Why can the deterministic program crash on a large sorted array in Python?

Recursion depth becomes n, and Python's default limit is about 1000, so it raises RecursionError. Randomized keeps depth near log n.

24. Solve the recurrence T(n) = 2T(n/2) + n.

By Master theorem (or recursion tree) T(n) = O(n log n).

25. Other ways to choose the pivot?

First, last, middle, random, median of three (first, middle, last).

26. Quick sort vs merge sort?

Merge sort: always O(n log n), stable, needs O(n) extra space. Quick sort: average O(n log n), worst $O(n^2)$, in-place and usually faster in practice.

27. Lomuto vs Hoare partition?

Lomuto: simple, pivot is last element, scans one direction. Hoare: two pointers from both ends, fewer swaps, a bit trickier.

28. How can recursion depth be reduced?

Recurse on the smaller part first and loop on the larger part (O(log n) stack), or use insertion sort for small sub-arrays.

29. What happens with all equal elements?

Simple Lomuto partition gives a one-sided split every time, so $O(n^2)$. Fix: three-way partition.

30. Where is quick sort used?

Library sort functions (C qsort, many language sorts), database and general-purpose sorting.

31. Advantages and disadvantages?

Advantages: fast on average, in-place, cache friendly. Disadvantages: $O(n^2)$ worst case, not stable, recursive.

32. What is divide and conquer?

Divide the problem into smaller subproblems, solve them, and combine the answers. Examples: quick sort, merge sort, binary search.

33. Base case of quick sort?

low >= high (zero or one element is already sorted).

Assignment 5 - N-Queens with first queen placed (Backtracking)

★ 1. What is backtracking? Give the general procedure.

Finding a solution step by step; if a step is wrong, come back and change it. Procedure: 1) start with a sub-solution, 2) check if it can lead to a solution, 3) if not, go back, change it and continue. Usually done by recursion.

★ 2. State the N-Queens problem and explain the solution.

Place N queens on an N x N board so that no two are in the same row, column or diagonal. Place a queen in a safe cell, then try the next queen. If no safe cell is left, go back and move the previous queen.

★ 3. Write the algorithm for N-Queens using backtracking.

1) Start in the leftmost column. 2) If all queens are placed, return true. 3) Try all rows in this column; if safe, place the queen and recurse for the next column. 4) If recursion returns true, return true. Else remove the queen (backtrack) and try the next row. 5) If no row works, return false.

★ 4. Why is it applicable to N = 4 and N = 8 only?

It is not only for 4 and 8. It works for every N except 2 and 3 (no solution). N = 1 is trivial. 4 and 8 are just common examples.

5. How many solutions? What are the 4-queen solutions?

4-queens: 2 solutions, (2, 4, 1, 3) and (3, 1, 4, 2). 8-queens: 92 solutions.

6. When does a queen attack?

When another queen is in the same row, column or diagonal.

7. Explain isSafe() in your code.

Checks the row on the left, the upper-left diagonal and the lower-left diagonal. Returns False if any queen is found.

8. Why only the left side is checked?

Queens are placed column by column from the left, so the right side is still empty.

9. Explain solveNQUtil().

If col >= N return True. For each row i: if isSafe, place the queen, call itself for col+1. If it returns True, return True. Else set board[i][col] = 0 (backtrack). If no row works, return False.

10. Which line is the backtracking step?

board[i][col] = 0 - removing the queen after failure.

11. How do you place the first queen yourself?

Ask the user for the row and column, put 1 on that cell first, and then run the same solve function for the remaining columns, skipping that column and treating that queen as already placed. Your manual code starts with an empty 4x4 board - be ready to change this.

12. Time complexity?

O(N!) in the worst case. Space $O(N^2)$ for the board.

13. Walk through 4-queen quickly.

Q1 at (1,1). Q2 at (2,3). Q3 has no place, so backtrack. Move Q2 to (2,4), Q3 to (3,2). Q4 has no place, backtrack again up to Q1. Move Q1 to (1,2) and continue until a solution is found.

14. Applications of backtracking?

N-Queens, sum of subsets, graph colouring, Hamiltonian cycle, maze, Sudoku.

15. Backtracking vs brute force?

Brute force tries all possibilities. Backtracking stops a wrong path early.

16. Backtracking vs Branch and Bound?

Backtracking uses DFS for feasible solutions. B&B is for optimisation and uses bounds to prune.

PART B - Machine Learning

Basic questions

1. What is machine learning?

Computers learn from data and make predictions without being programmed with every rule.

2. Types of machine learning?

Supervised (labelled data), unsupervised (no labels), reinforcement (reward based).

3. Regression vs classification?

Regression predicts a number (fare, price). Classification predicts a class (spam / not spam).

4. Overfitting and underfitting?

Overfitting: too good on training data, bad on new data. Underfitting: too simple, bad on both.

5. Why split into train and test?

To test the model on unseen data. Common split 80:20.

6. What is a dataset, feature, label?

Dataset = collection of rows. Feature = input column. Label/target = column to predict.

7. What is supervised vs unsupervised?

Supervised uses known answers (KNN, SVM, regression). Unsupervised finds groups without answers (K-means).

8. What is cross-validation?

Data is split in k parts; each part is used once for testing and the rest for training. The scores are averaged.

Assignment 7 - Uber fare prediction (Linear Regression and Random Forest)

★ 1. What is data preprocessing?

Preparing raw data so it is suitable for a model. Real data has noise, missing values, wrong formats. Steps: get dataset, import libraries, import data, find missing data, encode categorical data, split train/test, feature scaling.

★ 2. Define outliers.

Data points that are very different from the rest of the data. They can distort the model. Found using box plot or IQR; treated by removing or capping.

★ 3. What is linear regression?

A simple supervised algorithm that predicts a continuous value using a straight-line relationship between the dependent variable (y) and one or more independent variables (x).

★ 4. What is the Random Forest algorithm?

An ensemble of many decision trees built on different parts of the data. Final result is the average (regression) or majority vote (classification). More trees means higher accuracy and less overfitting.

★ 5. Explain pandas and numpy.

pandas: library for reading and handling tabular data (DataFrame, read_csv, dropna). numpy: library for fast arrays and maths.

6. What is a box plot?

Plot of minimum, first quartile, median, third quartile and maximum. Points outside whiskers are outliers.

7. What is the Haversine formula?

Gives the distance between two points on earth from latitude and longitude. Used to get trip distance.

8. What is correlation?

How strongly two columns move together, from -1 to +1. Shown with df.corr() and a heatmap.

9. What is Mean Squared Error (MSE)?

Average of squared errors. It is never negative and values close to zero are better. RMSE is the square root of MSE.

10. What is R2 score?

Fraction of variance explained by the model. Closer to 1 is better.

11. Which model is better here and why?

Usually Random Forest - handles non-linear data. State the scores from your own output.

12. Why is Linear Regression affected by outliers?

Because it squares errors, one extreme point pulls the line.

13. What did you do in preprocessing for Uber data?

Dropped unneeded columns, removed null values, converted date-time, calculated distance, removed outliers (wrong fares, passengers, coordinates).

14. What is matplotlib?

Python library for drawing graphs and plots.

15. Why do we use a heatmap?

To see correlation between all columns at once.

Assignment 8 - Email spam classification (KNN and SVM)

1. Describe the dataset.

5172 emails (rows) and 3002 columns. Column 1 = email name, last column = label (1 spam, 0 not spam), 3000 columns = word counts.

2. What is binary classification?

Classification with only two classes - here normal (not spam) and abnormal (spam).

3. Explain KNN.

Supervised algorithm. A new point gets the class of the majority of its K nearest labelled points. Example K = 5, 3 green + 2 red gives green.

4. Explain SVM.

Finds the best line (hyperplane) with maximum gap (margin) between two classes. The nearest points are support vectors.

5. Which is better, KNN or SVM?

SVM is usually more accurate and faster for 3000 features. Quote the accuracies from your output.

6. What is train-test split?

Divide data into training and test sets (e.g. 80/20). random_state keeps the split same each time.

7. Effect of K in KNN?

Small K: noisy, overfits. Large K: smooth, may underfit.

8. Distance used in KNN?

Euclidean distance.

9. What is a kernel in SVM?

A function that lets SVM separate non-linear data (linear, polynomial, RBF).

10. Need of preprocessing here?

Drop the name column, check missing values, split features and label, scale if needed.

11. Which metrics did you use?

Accuracy, confusion matrix, precision, recall.

12. Why precision matters for spam?

A good mail wrongly sent to spam (false positive) is costly.

13. Why is KNN slow at prediction?

It compares a new email with all training emails.

Assignment 9 - Bank customer churn (Neural Network)

★ **1. What is Normalization?**

Scaling values to a range, usually 0 to 1: (x - min) / (max - min). Makes all features comparable.

★ **2. What is Standardization?**

Scaling so that mean = 0 and standard deviation = 1: z = (x - mean) / std.

★ **3. Explain the confusion matrix.**

A table comparing actual and predicted classes: TP (true positive), TN (true negative), FP (false positive), FN (false negative).

★ **4. Define classification accuracy, misclassification rate, precision.**

Accuracy = (TP + TN) / total. Misclassification rate = (FP + FN) / total = 1 - accuracy. Precision = TP / (TP + FP).

★ **5. Give one example of a confusion matrix.**

TP = 50, TN = 35, FP = 5, FN = 10 (total 100). Accuracy = 85 percent. Misclassification = 15 percent. Precision = 50/55 = 90.9 percent. Recall = 50/60 = 83.3 percent.

6. What is an Artificial Neural Network?

A network of connected nodes (neurons) in layers that mimics the brain. Input layer, hidden layer(s), output layer.

7. Compare biological and artificial neuron.

Dendrites = inputs, cell nucleus = nodes, synapse = weights, axon = output.

8. What is Keras? What is TensorFlow?

TensorFlow is Google's deep learning library. Keras is a simple high-level API on top of it to build networks easily.

9. Steps of the assignment?

Read data, separate features and target, split train/test, normalise, build and train the model, improve it, print accuracy and confusion matrix.

10. What is your model design?

Dense layers with ReLU in hidden layers, one sigmoid output node, Adam optimizer, binary cross-entropy loss.

11. Why sigmoid in output?

It gives a probability between 0 and 1 for leave / stay.

12. What is an activation function?

A function that adds non-linearity to a neuron, e.g. ReLU, sigmoid, softmax.

13. Epoch, batch size?

Epoch = one full pass over training data. Batch size = samples used for one weight update.

14. Points of improvement?

More layers/neurons, more epochs, dropout, tuning learning rate, handling class imbalance.

15. Which columns are dropped?

RowNumber, CustomerId, Surname - they do not help prediction.

16. How are Geography and Gender handled?

Label encoding for Gender, one-hot encoding for Geography.

17. Dataset size?

10,000 rows, 14 features. Target = Exited (1 left, 0 stayed).

18. Why is accuracy alone not enough?

Most customers stay, so a model saying "stay" for everyone still gets high accuracy. Check precision, recall, F1.

19. What is backpropagation?

Method to update weights by sending the error backward through the network.

Assignment 10 - KNN on diabetes.csv

1. What is the objective?

Build KNN on diabetes data and compute confusion matrix, accuracy, error rate, precision and recall.

2. Describe the dataset.

768 rows (patients) and 9 columns: 8 input features (pregnancies, glucose, BP, BMI, insulin, age...) and 1 target Outcome.

3. Explain KNN.

Supervised model. It looks at the K closest labelled points and assigns the majority label.

4. What does train_test_split do? What is stratify?

Splits data (test_size 0.2 = 20 percent test). stratify = y keeps the same class ratio in train and test.

5. How did you choose K?

GridSearchCV tried K = 1 to 24 with 5-fold cross-validation. Best K = 14. Accuracy improved by over 4 percent.

6. What is GridSearchCV / hyper-parameter tuning?

Trying many parameter values automatically and picking the best by cross-validation.

7. Define accuracy, error rate, precision, recall.

Accuracy = (TP+TN)/total. Error rate = (FP+FN)/total. Precision = TP/(TP+FP). Recall = TP/(TP+FN).

8. Which is more important for diabetes - precision or recall?

Recall. We must not miss a sick patient.

9. Why scale data for KNN?

KNN uses distance, so large-range columns would dominate.

10. What is ROC curve and AUC?

ROC plots true positive rate vs false positive rate. AUC is the area under it: 1 = perfect, 0.5 = random.

11. Other metrics in objective?

confusion_matrix, accuracy_score, mean_squared_error, r2_score, roc_auc_score, roc_curve.

12. Random forest classification is also mentioned - what is it?

Many decision trees vote; the majority class is the final answer.

13. Limitations of KNN?

Slow on big data, needs scaling, sensitive to noise and irrelevant features.

14. Zero values in Glucose, BMI?

They are impossible values (missing). Replace with mean or median.

Assignment 11 - K-Means / Hierarchical clustering with elbow method

1. What is clustering?

Grouping unlabelled data so items in a cluster are similar and different from other clusters. It is unsupervised learning.

2. Uses of clustering?

Marketing (customer groups), real estate (property groups), bookstore/library management, document analysis.

3. Explain K-Means.

Unsupervised, centroid-based. 1) Choose K. 2) Pick K centres. 3) Assign each point to nearest centre. 4) Move each centre to the mean of its points. 5) Repeat until no change.

4. What is the elbow method?

Plot WCSS against K. Choose K at the bend (elbow) of the curve. In the manual K = 5.

5. What is WCSS?

Within-cluster sum of squares - sum of squared distances of points from their cluster centre.

6. Describe the dataset.

Mall customers: CustomerID, Gender, Age, Annual Income (k$), Spending Score (1-100).

7. Which features did you cluster?

Annual Income and Spending Score (2D), and then Age, Income, Spending Score (3D).

8. Advantages and problem of K-Means?

Simple, fast, scalable. Problem: need to choose K; weaker in high dimensions.

9. What is hierarchical clustering?

Builds a tree (dendrogram) by merging closest clusters (agglomerative) or splitting (divisive). No need to fix K first.

10. K-Means vs Hierarchical?

K-Means: faster, needs K. Hierarchical: slower, gives dendrogram.

11. Is clustering supervised?

No, unsupervised.

12. Meaning of the 5 clusters?

Low/high income combined with low/high spending, and a medium group. High income + high spending = best customers for offers.

13. What is a centroid?

The centre (mean) of a cluster.

PART C - Blockchain Technology

Basic questions

1. What is blockchain?

A shared digital ledger. Data is stored in blocks, each linked to the previous block by a hash. Hard to change.

2. What does a block contain?

Transactions, timestamp, nonce, previous block hash and its own hash.

3. Why is blockchain secure / immutable?

Changing one block changes its hash and breaks all later blocks. All nodes keep a copy.

4. What is a hash? Which is used?

A fixed-size fingerprint of data. Bitcoin uses SHA-256.

5. What is consensus? PoW vs PoS?

Rule for nodes to agree. PoW: miners solve a puzzle (high power). PoS: validators stake coins (less power).

6. What is a node? What is mining?

Node = computer in the network. Mining = solving the puzzle to add a block.

7. Bitcoin vs Ethereum?

Bitcoin is digital money (limited scripting, Turing-incomplete). Ethereum runs smart contracts (Turing-complete).

8. What is Ether? What is gas?

Ether is the currency of Ethereum. Gas is the fee unit for computation. Fee = gas used x gas price.

9. What is a dApp?

Decentralised application - a front end plus smart contracts on the blockchain.

10. What is a test network?

A practice network with free fake ETH (e.g. Sepolia).

11. What is Remix IDE?

Online tool to write, compile, deploy and test Solidity contracts.

12. What is the EVM?

Ethereum Virtual Machine - runs smart contract code.

13. What is a 51 percent attack?

One party controls more than half of the network and can cheat.

14. What are Wei, Gwei, Ether?

1 Ether = 10^{18} Wei. 1 Gwei = 10^{9} Wei.

Assignment 1 - Installation of MetaMask and spending Ether per transaction

1. What is MetaMask?

A software crypto wallet (browser extension and mobile app) to interact with Ethereum and decentralised apps. Developed by ConsenSys.

2. Steps to install MetaMask on Chrome?

1) Go to Chrome Web Store. 2) Search MetaMask. 3) Check number of downloads (to avoid clones). 4) Add to Chrome. 5) Click Get Started. 6) Create a Wallet (or Import Wallet). 7) Agree / No Thanks to data collection. 8) Create password and accept terms.

3. Why check the number of downloads?

Hackers make fake copies of MetaMask.

4. What does MetaMask do?

Stores account keys, sends and receives Ether and tokens, signs transactions, connects to dApps.

5. What is web3.js? Role in MetaMask?
A JavaScript library that MetaMask injects into every web page so sites can talk to Ethereum.

6. What is MetaMask Swaps?
Service that finds the best exchange rate across decentralised exchanges. Fee is 0.875 percent.

7. Why a new password if browser is changed?
MetaMask stores keys in the browser. Use Import Wallet with the secret phrase.

8. How do you see how much Ether a transaction costs?
MetaMask shows estimated gas fee before you confirm; Etherscan shows gas used and fee after.

9. What is gas fee and gas limit?
Fee = cost of processing a transaction. Gas limit = maximum you are ready to pay for one transaction.

Assignment 2 - Create your own wallet using MetaMask

1. How do you create a wallet?
Install extension, click Create a Wallet, agree or opt out of data collection, set password, save and confirm the secret backup phrase, wallet is ready (Account 1).

2. What is the secret recovery phrase?
Random words that can restore the wallet. Never share it.

3. What is a wallet address?
Your public Ethereum address shown below Account 1. Share it to receive funds.

4. Public key vs private key?
Public key/address: shared. Private key: secret, signs transactions.

5. How do you add funds?
Click View Account, copy the address, receive or buy ETH (test ETH from a faucet on a test network).

6. Is MetaMask hot or cold? Custodial?
Hot wallet (online) and non-custodial (you hold the keys).

7. Who created MetaMask?
ConsenSys. The manual also names Aaron Davis.

8. How can you adjust gas fees?
In the advanced tab you can change gas fee and gas limit.

9. Can you have many accounts?
Yes, many addresses can be created in one wallet.

Assignment 3 - Smart contract for bank account (deposit, withdraw, show balance)

1. What is a smart contract?
A program on the blockchain that automatically controls the transfer of digital assets when conditions are met. Works on IF-THEN logic. Has its own address.

2. Features of smart contracts?
Trust, accuracy, speed, backup, autonomy, safety, savings.

3. Use cases of smart contracts?
Real estate, vehicle maintenance, music royalties, government elections, management, healthcare payments, supply chain.

4. Bitcoin vs Ethereum contracts?
Bitcoin: Turing-incomplete, basic conditions. Ethereum: Turing-complete, custom contracts.

5. Explain your code.

pragma sets compiler ^0.4.19. owner is saved in the constructor as msg.sender. getOwner() returns owner. getbalance() returns owner.balance. deposit() is external payable and requires msg.value == 2 ether. withdraw() requires msg.sender == owner and transfers the whole contract balance (address(this).balance).

6. What is payable?

A function that can receive Ether.

7. What does require do?

Checks a condition; if false, the transaction is cancelled with the message.

8. What are msg.sender and msg.value?

msg.sender = caller address. msg.value = Wei sent with the call.

9. What is view?

A function that only reads data; it does not change state.

10. What is a constructor?

Runs once when the contract is deployed.

11. What is the difference between external and public?

external: called only from outside. public: from inside and outside.

12. Problem in this contract?

getbalance() shows the owner's wallet balance, not the contract balance. Deposit accepts only exactly 2 ether and keeps no per-customer record.

13. How do you run it?

Open Remix, paste code, compile, deploy (JavaScript VM or Injected MetaMask on Sepolia), set Value 2 ether and click deposit, then getbalance, then withdraw from the owner account.

Assignment 4 - Student data in Solidity (struct, array, fallback), gas and fees

1. What is Solidity?

Object-oriented language for writing smart contracts, mainly for Ethereum. Runs on the EVM. Developed by Christian Reitwiessner, Alex Beregszaszi and others.

2. Explain pragma.

Tells the compiler which version to use. pragma solidity >=0.4.16 <0.7.0 means version 0.4.16 up to below 0.7.0.

3. What is a state variable?

A variable stored permanently on the blockchain, e.g. uint public setData (uint = 256-bit unsigned integer).

4. Explain the Storage contract (set/get).

set(uint x) saves x in setData. get() is view and returns setData. Anyone can call set, so it is censorship-proof.

5. What is a structure?

A user-defined type that groups different variables, e.g. client_account (id, address, balance) or student (id, name, marks).

6. What is an array?

A list of same-type items. clients[] is an array of structs.

7. What is a fallback function?

A function with no name that runs when a call matches no function or when Ether is sent. Marked payable to accept Ether.

8. Explain the fallback code.

Contract fback has x = 10, mapping balance, SetX() and a payable fallback that adds msg.value to balance[msg.sender]. Contract Sender calls _receiver.transfer(100) to the fback address, so the fallback runs. (100 is Wei, not Eth.)

9. Explain the BankContract code.

Struct client_account and array clients. Manager is an address. Modifiers onlyManager and onlyClients. receive() accepts Ether. setManager, joinAsClient, deposit (clients only), withdraw(amount) sends amount x 1 ether, sendInterest (manager only) pays 1 ether to each client every 10 seconds, getContractBalance returns address(this).balance.

10. What is a modifier?

Reusable condition for functions. The line _; shows where the function body runs.

11. What is mapping?

Key-value store, e.g. mapping(address => uint). It cannot be looped.

12. Where do you see transaction fee and gas?

In Remix console after the transaction (gas, transaction cost, execution cost) and in MetaMask / Etherscan.

13. Transaction cost vs execution cost?

Transaction cost = total gas including base 21,000 and data. Execution cost = gas to run the code.

14. Why does deployment cost more than a function call?

Contract code is stored on the blockchain.

15. Storage vs memory?

Storage is permanent (costly). Memory is temporary.

16. What is now? What replaced it?

"now" gave the current time; it is removed in newer versions. Use block.timestamp.

17. How to reduce gas?

Avoid loops on big arrays, avoid extra storage writes, use mappings.

Assignment 5 - Survey of blockchain types and use cases

1. Types of blockchain?

Public, Private, Hybrid, Consortium.

Type	Who controls / joins	Advantages	Disadvantages	Examples (manual)
Public	Anyone; no central authority	Trustable, secure, anonymous, decentralised	Slow, high energy, slow adoption	Bitcoin, Ethereum
Private	One organisation; selected nodes	Fast, scalable, private, balanced	Less secure, centralised, few nodes	Hyperledger, Corda
Hybrid	Mix of public and private	Cannot be 51 percent attacked, cheap, customisable, choose participants	Less efficient, less transparent, no incentives	Ripple network, XRP
Consortium	Group of organisations (federated)	Fast, shared authority, privacy, flexible	Needs approval of all, can be corrupt, vulnerable	Tendermint, Multichain

2. What is permissionless blockchain?

Public, open to all, no central authority, fully open source, high transparency, heavy use of tokens. Con: low energy efficiency, low scalability, less privacy.

3. What is permissioned blockchain?

Closed network; only selected groups validate. Faster, customisable, strong privacy. Con: not truly decentralised, risk of corruption, owner can change rules.

4. Advantages of blockchain?
Trust, security, lower cost, speed, tokenization, immutable records, smart contracts.

5. Disadvantages of blockchain?
Data cannot be modified, needs large storage, lost private key cannot be recovered.

6. Real-life applications?
Voting, supply chain, healthcare, real estate, NFT marketplace, copyright, digital identity, immutable data backup, IoT.

7. Case study: Asset registry?
Ownership of land, property or vehicles stored on the blockchain - tamper proof and easy to verify.

8. Case study: Cryptocurrencies?
Digital money without a bank, e.g. Bitcoin and Ether, using blockchain for secure peer-to-peer transfers.

9. Case study: Interbank reconciliation?
Banks share one ledger so transactions match automatically, no manual reconciliation, faster settlement (e.g. Ripple).

10. Case study: Smart contracts?
Self-executing agreements, e.g. insurance payout, escrow, property transfer.

11. Case study: Supply chain traceability?
Track a product from origin to customer, e.g. food tracking with IBM Food Trust / Walmart.

12. Which blockchain would a company choose?
Private or consortium - for privacy, speed and known participants.

13. Use of hybrid blockchain?
Healthcare, government, real estate, finance - where some data is public and some must stay private.

Assignment 6 - e-Voting dApp (Ballot contract)

1. Explain the architecture of the Ballot contract.
Structs vote (voterAddress, choice) and voter (voterName, voted). Variables: countResult (private), finalResult, totalVoter, totalVote, ballotOfficialAddress, name, proposal. Mappings: votes (uint to vote, private) and voterRegister (address to voter). enum State {Created, Voting, Ended}.

2. Explain the modifiers.
condition(bool) - requires a condition. onlyOfficial - only the official address. inState(state) - function works only in that state.

3. Explain the constructor.
Takes official name and proposal. Sets the deployer (msg.sender) as ballot official and state = Created.

4. Explain addVoter().
Official adds a voter address and name in Created state; voted = false; totalVoter++.

5. Explain startVote() and endVote().
startVote moves Created to Voting. endVote moves Voting to Ended and stores finalResult = countResult. Only the official can call them.

6. Explain doVote().
Works in Voting state. If the sender has a name and voted == false: set voted = true, store the vote, if choice is true then countResult++, totalVote++, return true.

7. How is double voting stopped?
voted flag becomes true after the first vote and is checked before voting.

8. How are unauthorised people stopped?
Only registered voters (with a name) can vote; only the official can add voters and change the state.

9. Why use enum State?
To control the order: register voters, vote, end.

10. Is the vote secret?

No. Blockchain data is public. Real systems add extra cryptography.

11. What is a modifier's "_;" ?

Place where the function body runs after the check.

12. What does "struct = dictionary" mean?

A struct groups fields under names like a dictionary/record.

13. How do you test it?

Deploy with name and proposal, add voters from different accounts, startVote, vote true/false from each account, endVote, read finalResult, totalVote, totalVoter.

14. What is ABI?

Description of contract functions so a web page (web3.js) can call them.

15. Advantages of blockchain voting?

Tamper-proof, transparent, fast counting, no central point of failure.

16. Limitations?

Voter identity verification, privacy, scalability.

17. What is SPDX license identifier?

A comment at the top that states the licence (MIT) of the code.

PART D - Quick cheat sheet

Program	Time	Space
Fibonacci recursive	$O(2^n)$	O(n)
Fibonacci non-recursive / DP	O(n)	O(1) (O(n) with array)
Fibonacci matrix / fast doubling	O(log n)	O(log n)
Huffman coding	O(n log n)	O(n)
Fractional knapsack	O(n log n)	O(1)
Quick sort	Best/average O(n log n), worst $O(n^2)$	O(log n) average, O(n) worst
N-Queens (backtracking)	O(N!)	$O(N^2)$ board
KNN prediction	O(n x d) per query	O(n x d)
K-Means	O(n x K x iterations)	O(n + K)

Name	Formula
Fibonacci	Fn = Fn-1 + Fn-2, F0 = 0, F1 = 1
Quick sort	Best: T(n) = 2T(n/2) + n. Worst: T(n) = T(n-1) + n
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Misclassification rate	(FP + FN) / total = 1 - accuracy
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1	2 x P x R / (P + R)
Normalization	(x - min) / (max - min)
Standardization	(x - mean) / std
MSE / RMSE	mean of $(y - y_hat)^2$ / square root of MSE
R2	$1 - SS_{res} / SS_{tot}$
Gas fee	Gas used x gas price (1 Gwei = 10^{-9} ETH)

Technique	Idea	Used in
Greedy	Best local choice, never go back	Huffman, fractional knapsack
Dynamic Programming	Store solutions of overlapping subproblems	Fibonacci DP
Backtracking	Try, check, undo and try again (DFS)	N-Queens
Randomized	Random choice (pivot) to avoid bad inputs	Randomized quick sort
Divide and Conquer	Split into independent subproblems	Quick sort, merge sort, binary search

PART E - Mistakes in the manual (be careful)

Where	Manual says	Correct
DAA 1 theory	Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, **38**	Next term after 21 is 34.
DAA 1 table	DP with memoization space O(1)	Memoization needs O(n) space.
DAA 3 output	Maximum value = 24	It is 240 for W=50 (60,10), (100,20), (120,30).
DAA 5 question	Applicable to N = 4 and N = 8 only?	Works for all N except 2 and 3.
DAA 5 code	Pseudo-code upper-right diagonal uses k = k+1	Should be k = k-1. Code uses fixed N = 4 with no first-queen input.
ML 9 / 10	Assignment 10 conclusion talks about neural network	Assignment 10 is KNN on diabetes data.
ML 11	Title: sales_data_sample.csv	Code and description use Mall_Customers data.
BC 1	5,000,000+ users, 21 million users (Nov 2021)	Old figures.
BC 2	ConsenSys "founded by Aaron Davis"	ConsenSys was founded by Joseph Lubin; Aaron Davis is a MetaMask co-creator.
BC 3	getbalance() = owner.balance	Contract balance is address(this).balance.
BC 4	Comment "Transfers 100 Eth"	transfer(100) is 100 Wei.
BC 4 / 6	Uses "now" with ^0.6.6; Ballot uses ^0.7	"now" is removed in newer versions - use block.timestamp. Match Remix compiler to pragma.

Before the exam

- Run every program once and note the output (accuracy, R2, RMSE, gas used).
- Keep MetaMask ready on Sepolia with test ETH, and Remix open with contracts compiled.
- Keep all datasets downloaded (Uber, email spam, churn, diabetes, mall customers).
- Be ready for small changes: new Fibonacci n, new knapsack capacity, first queen position, new K in KNN / K-Means, new voter.